

Reader Overheating & Hardware Stress Predictor — How to Build It

A step-by-step guide written for someone starting from zero. What the project is, how to explain it to anyone in 60 seconds, and exactly what to build — phase by phase, with copy-paste code.

No new hardware needed

Uses existing FXR90 MQTT data

5 phases · ship value from Phase 2

Python + MQTT + SQLite

1 · Explain the project in 60 seconds

Memorise this page and you can present the project to anyone — manager, customer, or engineer.

The elevator pitch (say this)

"An FXR90 RFID reader already reports its own vital signs every few seconds — temperature, CPU load, memory, storage, radio and power status — over MQTT. Today nobody looks at that data until a reader fails. **This project listens to those vital signs continuously, turns them into a single 0–100 health score per reader, and raises an alert with a recommended action before the reader overheats or crashes** — instead of a technician discovering a dead reader after tags stopped being read."

The analogy (use this when someone looks confused)

It is a **fitness watch for the reader**. A fitness watch doesn't add new organs to your body — it just reads the pulse, temperature and activity you already produce, tracks what is *normal for you*, and warns you when something drifts. We do exactly that for each RFID reader: no new sensors, no new hardware — we simply pay attention to the health data the reader already broadcasts.

The problem → solution in one line each

Today	Readers run hot in warehouses; failures are discovered after reads stop — lost inventory data, emergency call-outs, unplanned downtime.
With this	A dashboard shows every reader's health score; a rising score triggers an alert like <i>"Reader dock-door-3: temperature trending up 40 minutes, CPU pinned — check ventilation / schedule reboot tonight."</i>

What it is NOT (say this too — it builds trust)

- It does **not** predict failures of external machines (motors, conveyors) — only the reader's own health.
- It needs **no firmware changes and no extra sensors** — only a program that subscribes to data the reader already sends.
- Machine learning is a **later** phase. The first shipped version is smart rules + per-reader baselines, which already delivers most of the value.

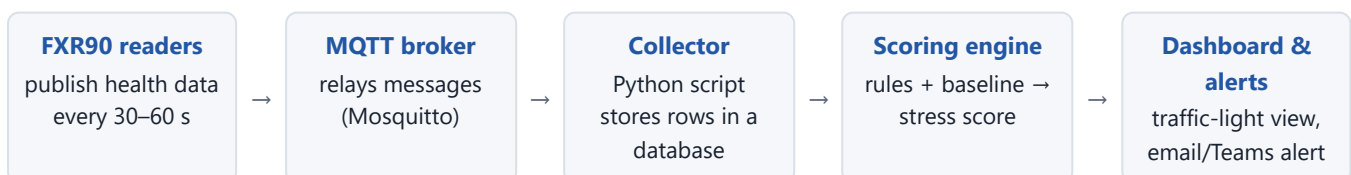
One honest framing that wins the room: "Phase 1–4 is a **real-time health monitor** — high confidence, ships in weeks. Prediction (ML) comes later, once we have collected enough history. We monitor now, predict later."

2 · The words you will need

Plain-language definitions. If you understand this page, you understand the whole project.

Term	Plain meaning
FXR90	Zebra's fixed RFID reader — a box mounted on a wall/dock door that reads RFID tags. It runs a small Linux computer inside, which is why it has CPU, RAM and temperature like a laptop.
MQTT	A lightweight messaging protocol used by IoT devices. Think of it as a group chat: the reader <i>publishes</i> messages to a named channel (a "topic"), and any program that <i>subscribes</i> to that topic receives them instantly.
Telemetry / heartbeat	The health report the reader publishes on a schedule (e.g. every 30–60 s): temperature, CPU, RAM, flash storage, radio and power status. "Heartbeat" = the regular ping proving the reader is alive.
get_status	The API command that returns the same health snapshot on demand — useful for testing before the heartbeat is configured.
Threshold rule	A simple IF-statement: <code>IF temperature > 70 °C THEN alert</code> . Easy to build, catches obvious problems, but knows nothing about what is "normal" for a specific reader.
Baseline	What is normal <i>for this particular reader</i> — e.g. "dock-door-3 normally sits at 48–55 °C on weekday afternoons." Learned automatically from 1–2 weeks of its own history.
Anomaly detection	Flagging behaviour that is unusual <i>compared to that reader's own baseline</i> , even if no fixed threshold is crossed. E.g. a reader that normally runs at 50 °C suddenly sitting at 63 °C — still below the 70 °C limit, but abnormal <i>for it</i> .
Stress score	Our single 0–100 number combining all signals, so a non-expert can rank readers by health without reading six graphs.
Labelled data	Historical records where we know the outcome ("this reader actually failed on March 3rd"). Machine learning that <i>predicts</i> failures needs this; most sites don't have it yet — which is exactly why ML is Phase 5, not Phase 1.
Broker	The MQTT "post office" (e.g. Mosquitto) that relays messages between the readers and our program. Free, open-source, runs on any PC/server.

The whole system in one picture



That's the entire architecture. Five boxes. Everything in this guide is just building those boxes left to right.

3 · The data you already have (verified in the API)

Every field below exists today in the FXR90 `get_status` response and the periodic heartbeat event. This was verified against the project's API schemas — nothing here is hypothetical.

Field	Type	What it tells you (maintenance meaning)
<code>temperature</code>	number (°C)	The headline signal. Sustained rise = blocked ventilation, failing environment cooling, or hardware stress. Trend matters more than the absolute value.
<code>cpu.system</code> / <code>cpu.user</code>	number (%)	Processor load. Pinned CPU for long periods = runaway app or overload → generates heat and slows tag processing.
<code>ram.used</code> / <code>free</code> / <code>total</code>	number	Memory pressure. Steady growth over days = memory leak → eventual crash/reboot.
<code>flash.platform.free</code> / <code>total</code>	number	Storage. A full flash partition can block logging and updates — a silent killer.
<code>radioActivity</code>	enum (on/off)	Whether the RFID radio is actively reading. Cross-reference: high temp while radio idle is more alarming than while at full blast.
<code>radioConnection</code>	enum	Radio subsystem connectivity — <i>disconnected</i> is an immediate red flag.
<code>powerSource</code> / <code>powerNegotiation</code>	enum	How the reader is powered (PoE/DC) and whether power negotiation succeeded. Failed negotiation = underpowered reader = flaky behaviour.
<code>uptime</code>	string	Time since last boot. A reader that quietly reboots itself every night is telling you something — uptime resets reveal it.

Known data-quality gaps (from this project's schema audit — fix or tolerate):

- `temperature`, `flash.platform.*` and `uptime` have **no description and no min/max bounds** in the schema — validate real values in Phase 1 before trusting them.
- The heartbeat interval is configurable (`managementEventConfig`) — **measure the actual cadence first**; every trend window below assumes ~1 sample/minute.
- `uptime` arrives as a string — parse it to seconds in the collector, not later.

What a raw heartbeat payload looks like (simplified)

```
{
  "temperature": 52.4,
  "cpu": { "system": 12.1, "user": 23.4 },
  "ram": { "total": 1024000, "used": 611200, "free": 412800 },
  "flash": { "platform": { "total": 4096000, "free": 1930000 } },
  "radioActivity": "on",
  "radioConnection": "connected",
  "powerSource": "poe+",
  "powerNegotiation": "success",
  "uptime": "12 days, 04:31:22"
}
```

One of these per reader per minute $\approx$ **1,440 rows/reader/day**. Fifty readers for a year is still only ~26 M small rows — SQLite or Postgres handles this without any big-data tooling.

4 · The stress score, explained simply

One number per reader so a non-expert can act. It is a weighted average of five sub-scores, each 0–100.

```
StressScore = 0.35 × temperature_score
              + 0.25 × compute_memory_score
              + 0.15 × flash_score
              + 0.15 × warning_error_score
              + 0.10 × power_radio_score
```

Sub-score	Weight	How to compute it (Phase 2 version)
Temperature	0.35	0 at ≤45 °C, 100 at ≥75 °C, linear in between; add +20 if the 30-min trend is rising ≥0.2 °C/min.
Compute + memory	0.25	Average of CPU% (system+user) and RAM-used%; add +20 if RAM usage has grown monotonically for 24 h (leak signature).
Flash	0.15	Flash-used%: 0 below 70 %, then ramp to 100 at 95 % full.
Warnings / errors	0.15	Count of error/warning conditions in the window (failed responses, restarts detected via uptime reset): 0 = none, 100 = ≥3.
Power + radio	0.10	0 if all nominal; 100 if <code>powerNegotiation</code> failed or <code>radioConnection</code> disconnected; 50 for degraded states.

Risk bands (what the user sees)

0–39 · LOW
Healthy — no action

40–64 · MEDIUM
Watch — check at next round

65–84 · HIGH
Act today — inspect / plan reboot

85–100 · CRITICAL
Act now — imminent failure risk

Be honest about the weights. 0.35 / 0.25 / 0.15 / 0.15 / 0.10 are **sensible starting guesses, not calibrated truth**. Say exactly that when presenting ("initial weights — we tune them against real incidents in Phase 4"). Anyone who asks "where do the numbers come from?" gets a straight answer, and you keep credibility.

Every alert must carry three things

- 1 **The score and what drove it** — "Score 78 (HIGH): temperature 68 °C and rising, CPU 91 % for 45 min."
- 2 **A likely reason** — "Pattern matches blocked ventilation or runaway application."
- 3 **A recommended action** — "Inspect airflow at dock-door-3; if clear, schedule reboot in tonight's window."

A score without a reason and an action is just a scary number — the action line is what makes this a *maintenance tool* rather than a graph.

Collect & store the data

Goal: health data from every reader flowing into one database table, reliably, for at least a week. No intelligence yet — plumbing first.

- 1 **Stand up an MQTT broker.** Install Mosquitto on any always-on machine: `apt install mosquitto` (or the Windows installer). Default port 1883.
- 2 **Point the readers at it.** In each FXR90's cloud/MQTT endpoint configuration, set the broker address and enable the **management events / heartbeat** interval (start with 60 s). Verify with `mosquitto_sub -h <broker> -t '#' -v` — you should see JSON heartbeats scroll past.
- 3 **Confirm the real cadence.** Watch for 10 minutes; note actual seconds between heartbeats per reader. All trend logic later depends on this number.
- 4 **Run the collector below.** It subscribes and writes every heartbeat to SQLite.
- 5 **Let it run for 7–14 days.** This history becomes each reader's baseline in Phase 3.

collector.py — complete, copy-paste

```
import json, sqlite3, time
import paho.mqtt.client as mqtt      # pip install paho-mqtt

DB = sqlite3.connect("reader_health.db", check_same_thread=False)
DB.execute("""CREATE TABLE IF NOT EXISTS health(
    ts REAL, reader TEXT, temp REAL, cpu REAL, ram_pct REAL,
    flash_pct REAL, radio_conn TEXT, power_neg TEXT, uptime_s INTEGER)""")

def uptime_to_seconds(s):             # "12 days, 04:31:22" -> seconds
    days, _, hms = s.partition(" days, ")
    h, m, sec = (hms or days).split(":")
    d = int(days) if hms else 0
    return d*86400 + int(h)*3600 + int(m)*60 + int(sec)

def on_message(client, userdata, msg):
    try:
        p = json.loads(msg.payload)
        reader = msg.topic.split("/")[0]      # adapt to your topic scheme
        ram = p["ram"]; flash = p["flash"]["platform"]
        DB.execute("INSERT INTO health VALUES(?,?,?,?,?,?,?,?,?)", (
            time.time(), reader, p["temperature"],
            p["cpu"]["system"] + p["cpu"]["user"],
            100 * p["ram"]["used"] / ram["total"],
            100 * (1 - flash["free"] / flash["total"]),
            p.get("radioConnection"), p.get("powerNegotiation"),
            uptime_to_seconds(p.get("uptime", "0:0:0"))))
        DB.commit()
    except (KeyError, ValueError) as e:
        print("skipped malformed payload:", e)    # never crash the collector

c = mqtt.Client()
c.on_message = on_message
c.connect("BROKER_IP", 1883)
c.subscribe("#")                               # narrow to your heartbeat topic once known
c.loop_forever()
```

Definition of done for Phase 1: `SELECT reader, COUNT(*) FROM health GROUP BY reader` shows a steadily growing row count for **every** reader, and a day-long gap-check finds no missing hours.

Rules engine — first real value

Goal: obvious problems raise an alert within one heartbeat. This alone catches a large share of real incidents — ship it and start building trust.

Starter rule set

Rule	Severity	Why / recommended action text
temperature > 70 °C	CRITICAL	Near hardware limit — "Inspect ventilation now; consider controlled shutdown."
temperature rising ≥ 0.2 °C/min for 30 min	HIGH	The <i>predictive</i> rule — catches trouble before the limit. "Check airflow / ambient temperature at the site."
CPU > 90 % for 15 min	HIGH	Runaway process. "Identify busy app; schedule reboot if persists."
RAM used > 90 %	HIGH	Crash risk. "Reboot in next maintenance window."
Flash > 90 % full	MEDIUM	"Purge logs / old packages before updates fail."
powerNegotiation = failure	CRITICAL	Underpowered reader. "Check PoE switch port budget / injector."
radioConnection = disconnected	CRITICAL	Not reading tags at all. "Power-cycle radio; escalate if repeats."
uptime decreased vs previous sample	MEDIUM	Silent reboot detected. "Review logs — was it planned?"
No heartbeat for 3 × interval	CRITICAL	Reader offline or network down. "Ping reader; dispatch if unreachable."

rules.py — the shape of it

```
def check(reader, rows):
    # rows = last 30 min from SQLite, newest last
    alerts, cur = [], rows[-1]
    if cur.temp > 70:
        alerts.append(("CRITICAL", f"temperature {cur.temp:.1f} C > 70",
            "Inspect ventilation now; consider controlled shutdown"))
    slope = (rows[-1].temp - rows[0].temp) / max((rows[-1].ts - rows[0].ts)/60, 1)
    if slope >= 0.2:
        alerts.append(("HIGH", f"temperature rising {slope:.2f} C/min over 30 min",
            "Check airflow / ambient temperature at the site"))
    if len(rows) >= 2 and cur.uptime_s < rows[-2].uptime_s:
        alerts.append(("MEDIUM", "reader rebooted (uptime reset)",
            "Review logs - was this planned?"))
    # ... one if-block per rule in the table above
    return alerts
```

Run it on a 60-second timer over the last 30 minutes of rows per reader. Deliver alerts however your team already communicates — email, Teams webhook, or just a log file at first.

Alert-fatigue rule (non-negotiable): one alert per reader per condition per 4 hours, max. The fastest way to kill this project is 50 duplicate e-mails on day one — people will mute it and never unmute it. Deduplicate from the very first version.

Per-reader baselines — the differentiator

Goal: catch problems that fixed thresholds miss, because "abnormal" depends on the reader. This is the layer competitors typically don't have — and it needs **no failure history**.

The idea in one example

Dock-door-3 has run at 48–55 °C every weekday afternoon for two weeks. Today it reads 63 °C. No fixed rule fires ($63 < 70$), but for *this reader* that is far outside normal — something changed: a blocked vent, a new heat source nearby, a stuck fan. Baselines catch it a day or two before the 70 °C rule would.

The method — rolling mean and z-score (no ML library needed)

- 1 For each reader × each metric, compute mean (μ) and standard deviation (σ) over the last 14 days, **per hour-of-day** (readers are warmer at 2 pm than 2 am — compare like with like).
- 2 For each new sample compute $z = (\text{value} - \mu) / \sigma$ — "how many standard deviations from this reader's own normal?"
- 3 Flag $z \geq 3$ **sustained for 15+ minutes** as an anomaly (sustained, so a single noisy sample never alerts).
- 4 Feed anomalies into the same alert pipeline, tagged "unusual for this reader" — with the baseline shown: *"normally 48–55 °C at this hour, now 63 °C."*

```
SELECT AVG(temp), -- plus stdev; SQLite: use extension or compute in Python/pandas
       CAST(strftime('%H', ts, 'unixepoch') AS INT) AS hour
FROM health
WHERE reader = ? AND ts > strftime('%s', 'now') - 14*86400
GROUP BY hour;
```

Why this beats jumping straight to ML: it is explainable ("3 σ above its own 2-week average" survives any meeting), needs zero labelled failures, works per-reader from day 15, and produces exactly the anomaly history you will later need to train real ML. It is the on-ramp to Phase 5.

Also update the stress score

Phase 2's temperature sub-score used fixed bands. Now blend in the baseline: `temp_score = max(fixed_band_score, min(100, z × 25))` — so a reader can go HIGH on abnormality alone, even below the fixed limits.

Dashboard, alert quality & the feedback loop

- 1 **Fleet view:** one row per reader — name, current stress score as a coloured chip (green/amber/orange/red), top driver ("temp trend"), last-seen time. Sort by score descending; the sick readers float to the top.
- 2 **Reader detail:** 7-day charts of temperature, CPU, RAM, flash with the baseline band drawn behind the line — anomalies become visually obvious.
- 3 **Alert log with outcome buttons:** every alert gets *Confirmed real* / *False alarm* / *Duplicate* buttons. **This is the most important feature on the page** — it builds the labelled dataset Phase 5 needs and measures whether people trust the system.
- 4 **Weekly precision review:** % of alerts marked "confirmed real". Target $\geq 70\%$. Below that, raise thresholds — a quiet, right system beats a loud, ignored one.

Tooling: Grafana (fastest, point it at the DB), or a small Flask/React page if you want the outcome buttons integrated. Start with Grafana + a separate outcomes form if speed matters.

PHASE 5 · MONTH 3+ · ONLY WHEN READY

Machine learning — the honest version

Do **not** start here. ML that predicts "failure in 1–6 h" needs labelled history that does not exist on day one.

- **Gate to enter Phase 5:** ≥ 3 months of telemetry *and* ≥ 20 –30 confirmed incidents from the Phase 4 outcome buttons. Fewer than that → stay on rules + baselines (they are already delivering value).
- **First model:** gradient-boosted classifier (XGBoost / LightGBM) on windowed features (means, slopes, z-scores over 30/60/180 min) predicting "incident within next 6 h". Boring and effective.
- **Evaluate on precision at fixed alert budget** (e.g. "at most 2 alerts/day fleet-wide, how many real incidents caught?") — never accuracy, incidents are rare so accuracy lies.
- Keep the rules engine running underneath forever — ML augments the safety net, it never replaces it.

Timeline at a glance

When	Milestone
Week 1–2	Broker + collector running; real heartbeat cadence measured; data accumulating.
Week 3	First shipped value: rules engine + deduplicated alerts.
Week 4–5	Per-reader baselines + z-score anomalies; stress score v2.
Week 6–7	Dashboard, outcome buttons, weekly precision review.
Month 3+	ML phase — only if the incident-label gate is met.

5 · Presenting it: script, demo, and the questions you will get

A 5-minute presentation plan

- 1 Problem (30 s):** "Readers fail silently; we find out when tag reads stop. Each surprise failure costs downtime and an emergency visit."
- 2 Insight (30 s):** "The reader already reports its vital signs every minute — nobody is listening." Show the raw JSON payload from page 3.
- 3 Solution (1 min):** The five-box diagram from page 2. "Listen → store → score → alert with a recommended action."
- 4 Demo (2 min):** Fleet view sorted by score; click the worst reader; show temperature trending up with its baseline band; show the alert text with the action.
- 5 Plan & ask (1 min):** The week-by-week table. "Working alerts in three weeks, no new hardware, ML later once we have history. I need N readers and a VM for the broker."

Questions you will get — and honest answers

"Is this just thresholds? We could do that in any monitoring tool."

Thresholds are only layer 1. Layer 2 learns each reader's personal normal and flags drift long before any fixed limit — that per-device baseline is what generic tools don't give you out of the box.

"Where does the data come from? Do we need to install anything on readers?"

No. The FXR90 already publishes health heartbeats over MQTT — we only configure the interval and subscribe. Zero firmware changes.

"Can it predict failures of our conveyor / motors too?"

No — and it doesn't claim to. It monitors the reader's own health. External machinery needs its own sensors; that would be a different project.

"How accurate is the prediction?"

Honest answer: phases 1–4 are detection and early-warning — high confidence. True hours-ahead prediction is Phase 5 and needs months of history plus confirmed incidents; we build the dataset for it from day one via the alert-outcome buttons.

"What if it cries wolf?"

Alert budget + dedup from day one, and a weekly precision review with a ≥ 70 % target. If precision drops, thresholds go up. The system earns attention; it doesn't demand it.

"What does it cost?"

Software is all open-source (Mosquitto, Python, SQLite/Postgres, Grafana). Cost is one small VM and the build time in the timeline table.

Success criteria to commit to (write these down before you start):

- ≥ 95 % heartbeat capture rate per reader per day (Phase 1)
- First true incident caught before user impact (Phase 2–3)
- Alert precision ≥ 70 % at the weekly review (Phase 4)

- ≥ 20 labelled incidents collected before any ML work starts (gate to Phase 5)

6 · Pitfalls, and your first-day checklist

The five ways this project fails (avoid them)

Pitfall	Prevention
Alert spam on week one	Dedup window + per-reader alert budget before the first alert is ever sent (page 6).
Trend logic built on assumed cadence	Measure the real heartbeat interval in Phase 1 step 3; recheck after any config change.
Trusting unvalidated fields	<code>temperature</code> / <code>flash</code> / <code>uptime</code> have schema gaps — eyeball a week of real values before wiring rules to them.
Starting with ML	Hold the Phase-5 gate: ≥ 3 months data, ≥ 20 confirmed incidents. Rules + baselines first.
No feedback loop	Outcome buttons from the first dashboard version — without labels you can never improve or prove value.

Day-one checklist

- 1 Pick 2–3 pilot readers (include the one with the worst reputation — early wins live there).
- 2 Install Mosquitto on a VM/PC that stays on; open port 1883 on the local network.
- 3 Configure MQTT endpoint + heartbeat (60 s) on the pilot readers; confirm with `mosquitto_sub`.
- 4 Run `collector.py`; check row counts after one hour, then after one day.
- 5 Create the shared alert channel (email group / Teams webhook) — empty for now.
- 6 Book the Phase-2 review for two weeks out: show live data, agree the rule thresholds with whoever knows the site.

What you need (complete list)

Item	Notes
FXR90 readers (2–3 pilot)	Network-reachable; admin credentials to set MQTT/heartbeat config.
One small VM / PC	Always-on; runs broker + collector + scoring + Grafana. 2 CPU / 4 GB is plenty for a pilot.
Software (all free)	Mosquitto, Python 3 + paho-mqtt, SQLite (Postgres later), Grafana.
People	One developer (this guide), plus a site/maintenance contact to sanity-check thresholds and confirm alert outcomes.
Reference docs	The concept deck (same folder), the FXR90 API schemas in this repo, and <code>project_resources/test_scenarios/</code> for ready-made config payloads.

Final framing to remember: this project succeeds by being **boring and trustworthy** — a monitor people believe, then a predictor people rely on. Monitor now, predict later.